THE ONE-LINER

A consistency model is the contract about what a read can observe after writes — and because replication is asynchronous, reads can be stale or out-of-order unless the system coordinates. The models run from linearizable (one copy, every read fresh) through causal and the session guarantees to eventual (converge, may be stale), and stronger costs latency and availability.

THE SPINE — WHAT YOU DRAW

1. **Why does having more than one copy create a consistency problem?**
   Because replication is asynchronous: a write commits on the primary and propagates to the other replicas afterward, so for a window they disagree and a read to a lagging replica returns a stale value (and concurrent writes to different replicas can conflict). You replicate for availability, read scaling, and locality — but each replica is a copy that can be behind. Eliminating the staleness needs synchronous coordination on every write, which sacrifices the latency and availability you replicated to gain — so more-than-one-copy forces the trade, and the consistency model names which side you took. Draw the second copy *first*: every anomaly on this board is "which copy answered, and how far behind was it."
2. **Draw the spectrum — and label each model by the anomaly it rules out, not by its name**
   Four boxes, strongest to weakest, each labelled with what it **forbids**. **Linearizable** — forbids the stale read: a completed write is visible to every read that starts after it (one copy, real-time order). **Causal** — forbids effect-without-cause: no reply visible without its message; unrelated concurrent writes may still be seen in either order. **Session guarantees** — forbid the two things one *user* notices: their own write vanishing (read-your-writes) and time running backwards on refresh (monotonic reads). **Eventual** — forbids nothing except permanent divergence: replicas converge, and that is the entire promise. Labelling by anomaly rather than by name is the whole trick, because it tells you which model to buy: match the cheapest model that forbids the bug you actually have.
3. **What does linearizable actually require — and why isn't "one leader" enough?**
   Write it as: *every operation appears to take effect atomically at some instant between its start and its finish, consistent with real time.* A single leader gives you a **total order** — but not linearizability, because a leader that has been **deposed during a partition and has not learned it yet** will happily serve reads from its own memory while the new leader is already accepting writes it has never seen. So draw the read path as: client to leader, and then a **quorum confirmation** before the leader is allowed to answer (etcd calls this ReadIndex). That extra round trip is the price of the guarantee. The tell of a shallow answer is "we have one primary, so we're strongly consistent"; the tell of a real one is "the node answering must *prove* it is still the leader."

4. **The session guarantees — draw where each one sits in the read path**
   Draw one user, one write to the primary, and a fan of replicas at different lag. **Read-your-writes**: their next read must not land on a replica behind their own write — so either pin them to the primary briefly, or carry the write's **LSN** and route to a replica that has replayed past it. **Monotonic reads**: their *successive* reads must not go backwards — so pin the session to one replica, or track the highest version seen and never serve from a replica behind it. **Monotonic writes**: their writes apply in issue order. **Writes-follow-reads**: their reply is ordered after the message it replied to. Mark the buying order on the board: read-your-writes first (users *retry* when it breaks, so it becomes a duplicate-data bug), monotonic reads the day you add a second read replica. And key the token to the **user**, not the device — phone-write, laptop-read is where the naive version breaks.
5. **Causal — what it preserves, and what it deliberately leaves loose**
   Draw two arrows and one non-arrow. Alice posts a message; Bob **reads it** and replies — that read creates a **dependency**, so the reply causally depends on the message, and no observer anywhere may see the reply without the message. Now draw Alice and Bob posting *unrelated* updates with no arrow between them: **concurrent**, so different observers may legitimately see them in either order. That is the deal — it preserves the ordering humans actually care about (cause before effect) without global real-time coordination, which is why it is the strongest model you can hold while staying **available under a partition**. Write the cost on the board too: every write must carry causal **metadata** and a replica must not apply it until its dependencies are applied — and that metadata is the reason most teams buy the session guarantees (causality scoped to one client, so it collapses to one token) instead of full causal.
6. **What do CAP and PACELC each tell you about the cost of strong consistency?**
   CAP: partitions are unavoidable, so *during* a partition you must choose consistency (refuse operations you cannot make safe — CP) or availability (serve possibly-stale responses — AP). PACELC completes it: *else*, with no partition, you still trade latency for consistency, because strong guarantees require coordination that adds latency to every operation. So strong consistency costs availability under a partition *and* latency all the time — which is why you buy it only where correctness demands it. Put a number next to the ELC arm, because that is the one that bites daily: sub-millisecond inside an AZ, but **75-90 ms** from us-east to eu-west, on every strongly-consistent write.
7. **Draw the quorum dial — R, W, N — and mark clearly what it does NOT give you**
   Draw N replicas, a write touching W of them, a read touching R of them, and shade the **intersection**. If **R + W > N** the sets must overlap, so a read observes at least one replica holding the latest *acknowledged* write — that is the whole guarantee, and N=3, R=2, W=2 is the standard tuning (fresh reads, tolerates one replica down). Now write the three things it does **not** buy, because this is the most over-claimed line in system design. It is **not linearizability**: a write that *failed* can still surface later via read-repair; concurrent writes have no defined order; and two successive reads can return newer-then-older. And a **sloppy quorum** (hinted handoff) voids the overlap entirely — the write is acked by nodes that are not even in the key's preference list. For a genuine compare-and-set you need a consensus round (Cassandra's Paxos LWT), not QUORUM.

8. **Two concurrent writes land on different replicas. Draw the three resolution paths.**
   One key, two writers, no coordination — draw three outgoing branches. **Last-write-wins**: compare wall-clock timestamps, keep the newer, **silently drop the other** — no error, no sibling, no log line — and note the failure on the board: with clock skew, the *fast-clocked* node wins regardless of what actually happened first, and a badly forward-skewed clock permanently shadows the key. **Version vectors**: neither version dominates, so the system *detects* genuine concurrency and surfaces **siblings** for the app to merge — nothing is lost, but you owe a merge. **CRDT**: the structure merges commutatively with no coordination at all (a G-counter sums per-replica slots, so both increments survive; a PN-counter is two of those and also handles decrements). Then draw the boundary line: a CRDT **cannot enforce a global invariant** — "balance never below zero," "one owner per username" — because two replicas can each locally approve a jointly-invalid state. That is exactly where you still pay for consensus.
9. **A user creates a record and immediately gets a 404. Draw the bug and the fix.**
   Draw it: write goes to the **primary**; the very next read is load-balanced to a **replica** that has not received it; the user gets "not found" on data they just created. Intermittent, worse under load, unreproducible locally — the classic read-your-writes violation. Before drawing a fix, draw the **measurement**: replication lag (`replay_lag` / `Seconds_Behind_Master`) charted against the 404 rate, plus *which* replica served the failure. Then two fixes. **Sticky-to-primary for a window** — cheap, and a guess: the window is a constant calibrated against a lag distribution that grows under load, during backfills, and while the replica vacuums, so it silently regresses. **LSN routing** — carry the write's log position with the *user* and serve the read only from a replica that has replayed past it, falling back to the primary. That one cannot expire, because correctness is derived from the actual replication position rather than a timer.

DECISIONS & SWITCH CONDITIONS

Strong (linearizable) → **Eventual** when fast, highly available, scales, survives partitions – but stale reads and concurrent-write conflicts the application must handle

Sticky-to-primary for a window → **LSN/GTID token** when carry the write's log position with the user and serve only from a replica that has replayed past it (fall back to the primary) – but you must thread the token through every read path and store it against the user, not the device

CP (consistency) → **AP (availability)** when keep serving reads and writes during a partition, so the system stays up – but replicas diverge and reads can be stale, needing reconciliation afterward

Last-write-wins → **Version vectors** when encode causality, so they genuinely detect concurrency and surface siblings – nothing is lost, but the application now owes a merge function and the metadata grows with the number of replicas

Strict quorum → **Sloppy quorum (hinted handoff)** when the home replicas are down and writes must keep succeeding through the failure – any available node accepts and hands it off later, but the acking nodes may have zero overlap with the read set, so the R + W > N guarantee is void until the hints drain

Regional authority → **Global consensus (Spanner-style)** when every write commits through a globally-distributed quorum, so you get strict serializability anywhere – but every read-write transaction pays the cross-region round trip plus a commit-wait, on the order of 100 ms+

One global level → **Per-operation** when matches cost to need – pay coordination only where freshness is required – but more surface area to reason about and get right

CEILINGS — THE NUMBERS

R + W vs N: 4 vs 3 overlap — R + W > N means every read set intersects every write set, so a read is guaranteed to observe the latest acknowledged write

Read freshness: fresh (not linearizable) quorums overlap — overlapping quorums guarantee a read sees the last ACKNOWLEDGED write — but this is NOT linearizability: a write that FAILED can still surface via read-repair, concurrent writes have no defined order, and a sloppy quorum voids the overlap entirely. A real compare-and-set needs Paxos, not QUORUM.

Write fault tolerance: 1 of 3 replicas may be down — a write needs W=2 acknowledgements, so it survives up to N - W = 1 replica failures before writes stall entirely

Read fault tolerance: 1 of 3 replicas may be down — a read consults R=2 replicas, so it survives up to N - R = 1 failures before reads stall

Quorum write latency: 80 ms — the write returns on the W-th acknowledgement. W=1 is satisfied by the local replica (~1 ms); W>1 must reach a peer, and if the quorum cannot form inside one region that peer is a full cross-region round trip

Coordination tax: 79 ms per write — the PURE network cost of the guarantee, paid on EVERY write with a perfectly healthy network — this is PACELC's ELC arm as a number, and it is exactly why you localize authority so the quorum forms inside one region

Serial updates to one key: 13 per second — each update must commit before the next can begin, so a single hot key is capped near 1000 / write-latency. You cannot raise this by adding replicas (that enlarges the quorum and makes it slower) — you raise it by sharding into more consensus groups

R + W > N gives overlapping quorums (fresh reads); a write needs W replicas up and a read needs R, so each tolerates N minus that many failures. But the write does not return until the **W-th acknowledgement** lands — so the moment the quorum cannot be formed inside one region, every strongly-consistent write pays a cross-region round trip. That is PACELC's "else" arm as an actual number, and it is the figure that ends the "why not make everything strong" argument.

TRAPS → THE FIX

"We'll make everything strongly consistent to be safe"	→	**Right-size per operation: strong for money, uniqueness, and locks; the weakest acceptable model (session guarantees or eventual) for everything else — ask "does any write decision depend on *this* read."**
"We use QUORUM reads and writes, so we're linearizable"	→	**Say what R + W > N actually buys — the read set intersects the write set, so you observe the latest *acknowledged* write — and then name the mechanism for real linearizability: a consensus round (Cassandra's Paxos lightweight transaction, a DynamoDB conditional write, an etcd CAS), not a QUORUM read.**

"Eventual consistency is fine for the account balance"	→	Use strong consistency (linearizable read, serializable transaction) for money and any operation where a stale or lost write is a correctness bug; reserve eventual for data where brief staleness is harmless.
"Read replicas, so reads always reflect the latest write"	→	Treat replica reads as eventually consistent: route reads that must reflect a user's own recent write to the primary (or a caught-up replica) for a short window — read-your-writes without paying for global strong consistency.
"We're CA — we don't really get partitions"	→	Say which one you chose *and why*, per data set: "the payment path is CP — under a partition it refuses rather than serving a stale balance; the feed is AP — it stays up and reconciles." Then use PACELC for the everyday cost, because that is where the latency actually goes.
"Last-write-wins, so the conflict is resolved"	→	Name the strategy honestly and pick it deliberately: **version vectors** when you must not lose a write (they detect genuine concurrency and surface siblings), **CRDTs** when the operation has a natural merge, and LWW only when losing a write is genuinely acceptable and you have said so out loud.
"It's eventually consistent, so I'll just increment the counter"	→	Model it as an *operation*, not a value: a **G-counter CRDT** (each replica increments its own slot; the value is the sum, merged by element-wise max) so both increments survive — and a **PN-counter**, which is two such maps with value = sum(P) - sum(N), once the count can also go down. Or an atomic increment against a single owner for that key (`INCR` , DynamoDB `ADD`).
"We're serializable, so a read always sees the latest commit"	→	Separate the axes: **serializability** is multi-object transaction *isolation*; **linearizability** is single-object real-time *recency*. What you want when a human is watching is **strict serializability** — both together — which is what Spanner provides.
"We put a cache in front — it doesn't change our consistency model"	→	State it plainly: the effective consistency model is the **weakest copy in the read path**, not the label on the database. So a read that feeds a write decision must never be served from the cache — and if it is served from a replica, a CDN, or a materialized view, that read is eventual too, whatever the database promises.

SENIOR TELLS — SAY THESE

- Match to the operation, and use the criterion that generalizes: **does any write decision depend on this read?** A stale read that renders pixels is harmless; a stale read that feeds an `if` that then writes is a correctness bug. Strong for money, uniqueness, locks and coordination; eventual for counts, feeds, profiles and recommendations — and tune it per call rather than globally.

- Start sticky if the system is small and the lag is comfortably inside the window — it is genuinely cheaper. Move to the token the moment you cannot state your p99 replication lag with confidence, because at that point the window is not a design, it is a guess. And whichever you pick, key it to the **user**: a phone-write followed by a laptop-read breaks any device-scoped implementation.
- CP for data where a wrong answer is unacceptable (payments, inventory, coordination, locks); AP for data where being up-but-stale beats being down (feeds, carts, presence) — and the choice can and should **differ per data set within one system**. Beware the third option people think they have: "CA" is not a choice, because a GC pause or an overloaded node is indistinguishable from a partition, so a system claiming CA is really CP or AP with the risk unexamined.
- Choose by what a lost write costs. LWW only when losing one is genuinely acceptable *and* you have said so out loud (a cached preference, a presence flag) — never for money or for anything a user typed. Version vectors when you must not lose a write and the app can merge. CRDTs when the operation has a natural merge (counters, sets, collaborative text) — and remember the boundary: "balance never negative" and "one owner per username" are not merge-able, so those still need consensus.
- This is a real fork and most people do not know they have taken it. Strict when a read must not miss an acknowledged write (anything on a correctness path). Sloppy when write **availability** is the product requirement and a temporarily-invisible write is survivable (a shopping cart, telemetry, a session). The unforgivable version is running sloppy while *claiming* R + W > N — the arithmetic still holds and the guarantee does not.
- Localize by default: the overwhelming majority of writes are for a key that "belongs" somewhere, so home-region authority gives you strong consistency at intra-region latency, which is the best trade in the topic. Pay for global consensus only for genuinely **global invariants** — a cross-region transfer, a globally-unique claim — and pay it knowing that a Spanner-class guarantee needs Spanner-class infrastructure (bounded-uncertainty clocks and a commit-wait), not just a config flag.
- Per-operation is the senior default (`QUORUM` vs `ONE`, primary vs replica routing, a lightweight transaction only where needed) — ask "what does *this* operation need." And make the choice **structural rather than advisory**: encode the guarantee in the API (`getBalanceForUpdate()` reads strongly and is the only way to read a balance on a write path; `getBalanceForDisplay()` is eventual and says so), because the real failure mode is a well-meaning engineer copying a fast eventual read into a debit path. Reserve a single global level for small systems where the simplicity genuinely is worth the over-pay.

HARDER ANGLES — CURVEBALL-READY

clock skew — Reason precisely about last-write-wins under clock skew — why the loss is *silent*, why tighter NTP does not fix it, and what the actual fix is.

monotonic-reads — Diagnose a pure read-path anomaly that no write bug explains, and fix it with the cheapest guarantee that removes it.

split-brain — Reason about split-brain — why majority quorum prevents it, why leases and pauses are the failure mode, and what to do once divergence has already happened.

the cache — Recognize that the consistency model is a property of the *read path*, not of the database — and that a cache is an unnamed replica.

the lock — Show that a lock requires linearizability, and then show that even a linearizable lock does not make the *protected resource* safe.

cross-shard — Recognize that sharding escapes coordination only *within* a shard, and that a cross-shard transaction reintroduces exactly the cost you sharded to avoid.

the invariant — Identify the hard boundary of CRDTs — they merge state, they cannot enforce a cross-replica invariant — without dismissing CRDTs generally.

read-repair — Explain the failed-write-becomes-visible anomaly in a Dynamo-style quorum store — and why it is not a bug, but the guarantee working exactly as specified.

IF THEY SAY "QUICKLY" — THE 30 SECONDS

Choose the weakest model each operation can tolerate. The criterion that generalizes is not "is this data important" — it is **"does any write decision depend on this read?"** A stale read that renders pixels is harmless; a stale read that feeds an `if` that then writes is a correctness bug. So: strong for money, uniqueness and locks; eventual for counts and feeds; read-your-writes for a user's own data — tuned per operation, because strong consistency is a premium paid on latency and availability, not a free safety default.